

Sprawozdanie z projektu

Verity Lens

System do ostrożnej weryfikacji informacji, wiarygodności newsów i
ryzyka obrazów AI

Autor: Artsiom Markevich

Kierunek / grupa: Informatyka Stosowana, grupa 1

Prowadzący: Jarosław Wilk

31 maja 2026

Repozytorium	Artsiom-beep/fake-news-detector
Backend publiczny	fake-news-detector-poi6.onrender.com
Wersja produktu opisana w raporcie	gałąź <code>main</code> po dodaniu rozszerzalnej bazy wiedzy
Tryb produktu	PC desktop, Web UI, REST API, Android APK, Render cloud

Contents

Streszczenie	4
1 Cel projektu	4
1.1 Cel główny	4
1.2 Cele szczegółowe	4
1.3 Ostateczny zakres produktu	5
2 Punkt wyjścia	5
2.1 Stan początkowy	5
2.2 Pierwsza diagnoza	6
3 Co się nie udało i dlaczego zmieniono podejście	6
3.1 Stary klasyfikator fake/real	6
3.2 Telefon przez LAN i tunele tymczasowe	6
3.3 Niejasny status wersji na telefonie	7
3.4 Błędy narzędzia Codex Desktop	7
3.5 Pełny detektor obrazów AI	7
3.6 Utrata metadanych na Androidzie	8
3.7 Zbyt zachowawcze odpowiedzi na proste fakty	8
3.8 Zmiana modułu Facts w kierunku bazy wiedzy	9
3.9 Zbyt literalne rozumienie krótkich faktów	9
3.10 Ocena newsów z dużych źródeł	10
4 Nowe podejście projektowe	10
4.1 Jeden kanoniczny silnik	10
4.2 Zasada ostrożności	10
4.3 Baza wiedzy i rozumowanie źródłowo-taksonomiczne	11
4.4 Oddzielenie news credibility od fact-checkingu	11
4.5 Jak działa analiza obrazów AI	12
5 Architektura końcowa	13
5.1 Warstwy systemu	13
5.2 Przepływ żądania	13
5.3 Kontrakt API	13
6 Implementacja interfejsów	14
6.1 Web UI	14
6.2 Aplikacja desktopowa Windows	14
6.3 Aplikacja mobilna Flutter	15
6.4 Wybór oryginalnych zdjęć na Androidzie	15
7 Wdrożenie chmurowe	16
7.1 Render	16
7.2 Dlaczego chmura była potrzebna	16

8 Testy i dowody działania	17
8.1 Testy automatyczne	17
8.2 Acceptance benchmark	17
8.3 Dowód działania na fizycznym telefonie	17
8.4 Paczka oddania	18
9 Historia prac	18
10 Najważniejsze decyzje projektowe	19
10.1 Nie używać starego ML jako głównego produktu	19
10.2 Nie obiecywać globalnej dokładności 95% bez benchmarku	19
10.3 Traktować telefon jako klienta, nie jako backend	19
10.4 Oddzielić demo od produkcji	19
10.5 Usunąć Screenshots z UI	20
10.6 Nie udawać pełnego detektora AI obrazów	20
10.7 Nazwać Facts bazą wiedzy	20
10.8 Użyć NLI w faktach, ale nie robić z niego jedynego sędziego	20
11 Aktualny sposób działania	20
11.1 Na komputerze	20
11.2 Na telefonie	21
11.3 W repozytorium	21
12 Ograniczenia	21
13 Możliwe dalsze prace	22
14 Wnioski	23
A Załącznik A: Komendy reprodukcyjne	23
B Załącznik B: Najważniejsze artefakty	23
C Załącznik C: Skrócony opis dla uruchomienia	24

Streszczenie

Celem projektu było zbudowanie praktycznego systemu do weryfikacji informacji, który można uruchomić na komputerze, w przeglądarce oraz na telefonie. Projekt rozpoczął się jako klasyczny *fake news detector*, ale w trakcie prac okazało się, że prosta klasyfikacja *fake/real* jest niewystarczająca i często nieuczciwa wobec użytkownika. Ostateczny produkt, nazwany *Verity Lens*, nie próbuje zgadywać za wszelką cenę. System zwraca twardy werdykt *true* lub *fake* tylko wtedy, gdy posiada silną podstawę: prostą stabilną wiedzę, jawny werdykt fact-checkingu albo jednoznaczne reguły. Dla zwykłych artykułów informacyjnych zwracana jest ocena wiarygodności, a dla obrazów ocena ryzyka na podstawie metadanych i śladów generatorów AI, nie absolutny dowód pochodzenia obrazu. W końcowej wersji moduł obrazów został świadomie ograniczony do analizy pliku: EXIF, nazw plików, markerów generatorów, wymiarów i słabych cech statystycznych. Pełnego rozpoznawania AI na podstawie samych pikseli nie udało się zrealizować w pełnym zakresie w darmowym środowisku chmurowym bez ryzyka fałszywych oskarżeń. W finalnej wersji wzmocniono także moduł *Facts*: krótkie twierdzenia są interpretowane przede wszystkim jako praca z bazą wiedzy. Baza zawiera stabilne fakty lokalne, może być rozszerzana przez użytkownika, a dopiero poza nią system korzysta z oficjalnej terminologii zdrowotnej WHO, streszczeń Wikipedii, reguł taksonomicznych oraz opcjonalnej warstwy NLI porównującej twierdzenie z fragmentem dowodu.

Raport opisuje cały przebieg projektu: stan początkowy, problemy ze starym modelem ML, porządkowanie architektury, zmianę podejścia na jeden kanoniczny silnik, budowę API, Web UI, aplikacji desktopowej Windows, aplikacji Flutter, wdrożenie Render, testy na telefonie, końcowe uproszczenie interfejsu przez usunięcie osobnej zakładki *Screenshots* oraz poprawę wyboru oryginalnych zdjęć z aparatu na Androidzie. Układ raportu oparto na typowej strukturze sprawozdania projektowego: cel, zakres, metoda, implementacja, wyniki, problemy, ograniczenia i wnioski.

1 Cel projektu

1.1 Cel główny

Celem głównym było przygotowanie działającego produktu, który pomaga użytkownikowi ocenić, czy informacja jest prawdziwa, fałszywa, niepewna albo czy artykuł wygląda wiarygodnie. Produkt miał być możliwy do pokazania w praktyce: lokalnie na PC, jako paczka desktopowa Windows, jako API, jako aplikacja mobilna oraz jako rozwiązanie działające przez internet bez konieczności trzymania komputera włączonego.

1.2 Cele szczegółowe

- uporządkować projekt po wcześniejszych eksperymentach ML i usunąć konflikt wielu wejść produktowych;
- stworzyć jeden kanoniczny silnik decyzyjny dla CLI, API, UI, desktopu i telefonu;
- rozdzielić zwykle sprawdzanie faktów od oceny wiarygodności newsów;
- poprawić inteligencję krótkich faktów bez przechodzenia w zgadywanie;
- uniknąć fałszywej pewności tam, gdzie brakuje źródeł;

- uruchomić backend lokalnie i w chmurze Render;
- zbudować aplikację mobilną Flutter z publicznym API;
- zbudować wersję Windows portable;
- przygotować testy automatyczne i paczkę oddania z dowodami działania.

1.3 Ostateczny zakres produktu

W aktualnej wersji użytkownik widzi trzy główne moduły:

- **News** – ocena wiarygodności artykułu lub adresu URL;
- **Facts** – sprawdzanie krótkiego twierdzenia w bazie wiedzy oraz możliwość dodawania nowych faktów;
- **Images** – analiza metadanych obrazu i ocena ryzyka, że plik zawiera ślady generatora AI albo nie pochodzi bezpośrednio z aparatu.

Funkcja OCR dla zrzutów ekranu pozostała w backendzie oraz w testach regresyjnych jako `screenshot_ocr`, ale nie jest już osobną zakładką w interfejsie użytkownika. Decyzja ta została podjęta po testach użyteczności: zakładka *Screenshots* wyglądała jak pełnoprawna funkcja dla użytkownika, a w praktyce była mniej jasna niż podstawowe moduły i zwiększała zamieszanie w aplikacji mobilnej.

2 Punkt wyjścia

2.1 Stan początkowy

Na początku projekt był mieszanką kilku podejść:

- archiwalny model ML do klasyfikacji *fake news*;
- prototypowe pliki predykcji i treningu;
- kilka potencjalnych punktów wejścia API;
- brak jednego jasnego kontraktu dla aplikacji mobilnej;
- brak pewności, która część kodu jest produktem, a która eksperymentem.

Największym problemem nie był brak kodu, tylko brak jednej prawdy architektonicznej. Istniał stary model, który mógł wyglądać atrakcyjnie jako “AI detector”, ale był zbyt ciężki, słabo kontrolowalny i nie nadawał się jako pewny produkt do demonstracji. Pojawiał się też ryzykowny problem semantyczny: system mógł zachowywać się jak klasyfikator, który na siłę wybiera **fake** albo **real**, nawet gdy dane nie dawały do tego podstaw.

2.2 Pierwsza diagnoza

Po analizie projektu przyjęto trzy założenia:

1. produkt musi mieć jeden kanoniczny silnik, a nie kilka konkurencyjnych ścieżek;
2. wynik *uncertain* jest poprawnym wynikiem, jeżeli system nie posiada mocnych dowodów;
3. zwykły artykuł newsowy nie jest tym samym co twierdzenie sprawdzane przez fact-checking.

To zmieniło kierunek prac. Zamiast ulepszać stary klasyfikator, projekt został przebudowany wokół ostrożnego pipeline'u, który potrafi powiedzieć: "nie wiem wystarczająco dużo".

3 Co się nie udało i dlaczego zmieniono podejście

3.1 Stary klasyfikator fake/real

Pierwotny kierunek opierał się na myśleniu: użytkownik podaje tekst, a model zwraca *fake* albo *real*. To podejście nie spełniało wymagań produktu z kilku powodów:

- model był ciężki i niewygodny do pakowania w desktop oraz chmurę;
- wynik klasyfikatora nie wyjaśniał użytkownikowi, na czym opiera się decyzja;
- system nie odróżniał artykułu newsowego od pojedynczego twierdzenia;
- fałszywa pewność była większym ryzykiem niż ostrożna odpowiedź *uncertain*.

Dlatego stary kod ML został oddzielony od runtime'u produktu. Nie został bezmyślnie usunięty, lecz przeniesiony do archiwum, aby zachować historię i możliwość analizy:

- `archive/legacy_multimodal/`
- `archive/legacy_factcheck_v1/`
- `docs/legacy_ml_inventory.md`

3.2 Telefon przez LAN i tunele tymczasowe

Na etapie mobilnym początkowo wystarczało, że telefon łączył się z komputerem w tej samej sieci Wi-Fi albo przez tymczasowy tunel. To działało do demonstracji, ale nie rozwiązywało głównego celu: telefon miał działać bez komputera. Tryb LAN wymagał, aby PC był włączony, API działało lokalnie, firewall przepuszczał ruch, a telefon był w tej samej sieci. Tunel tymczasowy również był niewystarczający, bo adres mógł wygasnąć.

Rozwiązaniem stało się wdrożenie backendu na Render i budowa APK z osadzonym publicznym adresem HTTPS:

`https://fake-news-detector-poi6.onrender.com`

3.3 Niejasny status wersji na telefonie

W trakcie testów pojawił się praktyczny problem: po zbudowaniu nowej aplikacji telefon nadal potrafił pokazywać stary interfejs. W szczególności po usunięciu zakładki *Screenshots* na ekranie nadal była widoczna stara wersja. Problem rozwiązano przez:

- czyszczenie buildu Flutter poleceniem `flutter clean`;
- ponowne pobranie zależności;
- czystą kompilację APK release;
- odinstalowanie starego pakietu z telefonu;
- ponowną instalację aktualnego `VerityLens-cloud.apk`;
- sprawdzenie pakietu `app.veritylens.mobile` przez ADB.

3.4 Błędy narzędzia Codex Desktop

Podczas prac pojawiały się okna błędu typu *Unable to find Electron app*. Po analizie uznano, że nie był to błąd projektu Verity Lens, backendu, Pythona, Fluttera ani telefonu. Był to problem środowiska narzędziowego aplikacji Codex Desktop, prawdopodobnie związany z obsługą wewnętrznych akcji `click/action`. Nie wpłynęło to na produkt, ale wprowadzało zamieszanie podczas pracy.

3.5 Pełny detektor obrazów AI

Najtrudniejszym elementem okazała się funkcja rozpoznawania obrazów AI. Na początku naturalnym pomysłem było zrobienie prostego przycisku, który odpowiada:

to jest obraz AI / to nie jest obraz AI

W praktyce taki wynik byłby zbyt mocną obietnicą. W pełnym zakresie wymagałby ciężkiego modelu wizualnego, dużego benchmarku, kalibracji na wielu typach obrazów oraz stabilnego środowiska obliczeniowego. Darmowy backend Render ma ograniczenia pamięci i czasu startu, więc uruchamianie dużych modeli typu Transformers/Torch byłoby wolne, niestabilne albo zbyt kosztowne.

Próbowano przewidzieć opcjonalną ścieżkę modelową przez zmienną `FACTCHECK_AI_IMAGE_MODEL`. Po testach przyjęto jednak ostrożną zasadę: sam wynik modelu wizualnego nie powinien automatycznie oskarżać obrazu o bycie AI, jeśli nie ma dodatkowych mocnych śladów. W przeciwnym razie aplikacja mogłaby prawie każdy nietypowy obraz oznaczać jako *Possible AI* albo zwracać myląco pewne wyniki. Dlatego w darmowej wersji chmurowej nie uruchomiono dużego modelu Torch. Zamiast tego backend wykonuje szybką analizę hybrydową: metadane, markery generatorów, nazwę pliku oraz lekkie cechy wizualne obliczane z pikseli.

Ostateczny moduł *Images* nie jest więc pełnym sądowym detektorem AI. Jest narzędziem do analizy ryzyka na podstawie tego, co plik sam o sobie ujawnia:

- czy w metadanych występują markery generatorów, np. Stable Diffusion, Midjourney, DALL-E, ComfyUI, Firefly;

- czy nazwa pliku zawiera ślady generatora, np. `ai-generated`, `midjourney`, `dalle`;
- czy obecne są oryginalne metadane aparatu, np. `Make`, `Model`, `DateTimeOriginal`, `LensModel`;
- czy metadane zostały utracone, co może oznaczać eksport, pobranie, edycję albo przesłanie przez komunikator;
- czy wymiary obrazu przypominają popularne formaty generatorów, np. kwadrat 1024x1024;
- czy lekkie cechy wizualne obrazu wskazują nietypowy wzorec: gładkość, brak naturalnego szumu, kompresję blokową, korelację kanałów i format typowy dla generatorów.

To podejście jest mniej efektywne niż pełny detektor AI, ale jest uczciwsze. Gdy system znajduje metadane generatora, traktuje to jako mocną wskazówkę. Gdy znajduje oryginalne metadane aparatu, pokazuje *Camera metadata found*, ale nadal nie twierdzi, że obraz na pewno jest autentyczny. Gdy metadanych brakuje, system nadal sprawdza lekkie sygnały wizualne. Jeżeli są mocne, pokazuje *AI signals found*; jeżeli są słabe, pokazuje *Possible AI signals* albo *Weak file clues*. Brak EXIF sam w sobie nie jest jednak dowodem na AI.

3.6 Utrata metadanych na Androidzie

Podczas testów na fizycznym telefonie okazało się, że wybór obrazu przez standardową Galerię lub aplikację Zdjęcia często nie przekazuje oryginalnego pliku z aparatu. Telefon zwracał kopię w katalogu typu `/sdcard/Pictures/`, np. plik o nazwie:

`JPEG_20260531_220216_...jpg`

Taki plik miał mały rozmiar i nie zawierał oryginalnego EXIF. W efekcie backend poprawnie odpowiadał *Metadata missing*, mimo że użytkownik naprawdę zrobił zdjęcie telefonem. Problem nie leżał w samym algorytmie, tylko w ścieżce wyboru pliku.

Najpierw próbowano wymusić systemowy picker dokumentów Androida i dopasowywać nazwy eksportowanych kopii do oryginałów w `DCIM/Camera`. Nie było to wystarczająco stabilne, bo Android nadal potrafił otwierać menu z Galerią, Google Photos albo innym eksploratorem. Końcowe rozwiązanie polegało na usunięciu tego zewnętrznego wyboru z głównego scenariusza. Aplikacja Flutter wyświetla własny dolny panel *Camera originals*, a natywna część Androida sama pobiera listę oryginalnych zdjęć z `MediaStore` i `DCIM/Camera`. Dzięki temu użytkownik wybiera plik aparatu bez przechodzenia przez aplikacje, które tworzą kopie i usuwają metadane.

3.7 Zbyt zachowawcze odpowiedzi na proste fakty

Jednym z praktycznych przykładów był fakt typu:

Books are made out of paper.

System początkowo odpowiadał zbyt ostrożnie, mimo że dla takiego stabilnego, codziennego faktu można zastosować lokalną wiedzę i prostsze reguły. Dodano obsługę typowych faktów materiałowych, dzięki czemu proste twierdzenia nie muszą przechodzić przez pełny ciężki pipeline źródłowy.

3.8 Zmiana modułu Facts w kierunku bazy wiedzy

Po wielu testach praktycznych okazało się, że próba zbudowania “uniwersalnego rozumu” dla każdego krótkiego zdania jest zbyt ryzykowna. System mógł znaleźć przypadkowy artykuł fact-checkingowy z podobnym słowem i na tej podstawie wydać błędny werdykt. Przykładem były zdania o książkach lub wodzie, które przez podobieństwo słów mogły zostać powiązane z niepasującymi materiałami z serwisów politycznego fact-checkingu.

Dlatego finalnie moduł *Facts* został opisany uczciwiej jako baza wiedzy z dodatkowymi źródłami. Wbudowana baza przechowuje stabilne fakty codzienne, np. o zwierzętach, materiałach, kolorach, prostych relacjach, stolicach i arytmetyce. Dodano także funkcję dopisywania faktów przez użytkownika. W aplikacji mobilnej i w wersji PC w sekcji *Facts* znajduje się zwijany blok *Knowledge base*, w którym można podać:

- podmiot, np. *tomato*;
- właściwość, np. *fruit*;
- czy fakt ma być zapisany jako prawdziwy czy fałszywy.

Backend zapisuje takie wpisy w pliku `outputs/factcheck_runs/user_knowledge_base_v1.json` i ładuje je razem z bazą wbudowaną. Dzięki temu podczas demonstracji można dodać nowy fakt, a następnie od razu sprawdzić zdanie o tym fakcie. To nie udaje pełnej inteligencji ogólnej, ale daje kontrolowany, wyjaśnialny i rozszerzalny mechanizm.

3.9 Zbyt literalne rozumienie krótkich faktów

Po wdrożeniu mobilnym wyszedł kolejny praktyczny problem: backend potrafił znaleźć właściwe streszczenie źródłowe, ale nie zawsze umiał wyciągnąć z niego konsekwencję logiczną. Przykładowo dla zdania *covid is virus* system widział opis COVID-19, ale zwracał *uncertain*, ponieważ literalnie szukał słowa *virus* jako dokładnej właściwości choroby. Podobnie zdania typu *Mars is a star*, *Whales are fish* albo *Paris is the capital of Germany* były wcześniej zbyt często klasyfikowane jako niepewne.

Rozwiązaniem nie było dodawanie pojedynczych wyjątków dla każdego pytania. Dodano ogólną warstwę rozumowania źródłowo-taksonomicznego:

- oficjalną regułę WHO dla relacji COVID-19 i SARS-CoV-2;
- rozpoznawanie klas niezgodnych, np. *planet* kontra *star*, *mammal* kontra *fish*, *arachnid* kontra *insect*;
- prostą hierarchię klas, np. *arachnid* oznacza także *animal*;
- relację *capital of X*, dzięki której system może odróżnić Francję od Niemiec;
- lepszą obsługę liczby mnogiej i wariantów słów, np. *gas/gases*, *virus/viruses*.

Po dalszych testach okazało się jednak, że same reguły nadal są za słabe dla wielu zwykłych twierdzeń. Reguła dobrze obsługuje zaplanowane klasy, ale nie rozumie każdego zdania naturalnego. Dlatego moduł *Facts* otrzymał opcjonalną warstwę NLI (*Natural Language Inference*). Backend może załadować model `Xenova/mobilebert-uncased-mnli` w wariancie ONNX przez zmienne `FACTCHECK_NLI_MODEL`, `FACTCHECK_NLI_BACKEND` i `FACTCHECK_NLI_ONNX_FILE`. Model nie generuje odpowiedzi jak chatbot. Jego zadanie

jest węższe i bezpieczniejsze: ocenić, czy znaleziony fragment źródła wspiera twierdzenie, przeczy mu, czy jest neutralny. Taki wynik jest używany tylko wtedy, gdy źródło jest zaufane, dopasowanie do twierdzenia jest wystarczająco mocne, a nie ma istotnego kontrdowodu. W przeciwnym razie system nadal zwraca **uncertain**.

3.10 Ocena newsów z dużych źródeł

Dla zwykłych artykułów BBC lub podobnych źródeł system początkowo bywał zbyt nieśmiały. Problem polegał na tym, że brak wielu niezależnych potwierdzeń nie powinien automatycznie obniżać wiarygodnego artykułu do słabej oceny. Wprowadzono model `source_quality_corroboration_v2`, który osobno traktuje:

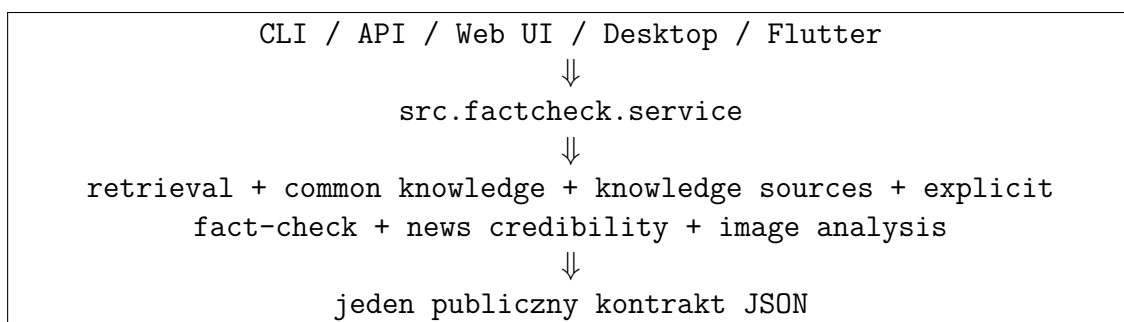
- jakość źródła;
- jakość samego artykułu;
- niezależne potwierdzenia;
- flagi ryzyka.

Dzięki temu dobry artykuł z wiarygodnej redakcji może otrzymać ocenę wysoką albo średnią bez udawania, że system dowiódł prawdziwości każdej informacji zawartej w artykule.

4 Nowe podejście projektowe

4.1 Jeden kanoniczny silnik

Najważniejszą zmianą architektoniczną było utworzenie jednej ścieżki produktu:



W praktyce oznacza to, że aplikacja na telefonie nie ma własnej logiki decyzyjnej. Telefon wysyła żądanie do API, a wynik jest liczony przez ten sam backend, którego używa Web UI i desktop. Zmniejsza to ryzyko, że różne wersje produktu będą odpowiadały inaczej na to samo pytanie.

4.2 Zasada ostrożności

W projekcie przyjęto zasadę:

Jeżeli system nie ma wystarczających dowodów, powinien powiedzieć **uncertain**, a nie zgadywać.

To jest decyzja produktowa, nie tylko techniczna. System do weryfikacji informacji może zaszkodzić użytkownikowi, jeżeli zbyt łatwo nada etykietę *fake*. Dlatego *Verity Lens* ma trzy poziomy odpowiedzi:

- **true** – mocne potwierdzenie lub stabilna wiedza;
- **fake** – mocne obalenie lub jawny werdykt fact-checkingu;
- **uncertain** – brak wystarczającego dowodu.

4.3 Baza wiedzy i rozumowanie źródłowo-taksonomiczne

Finalny moduł *Facts* jest traktowany jako baza wiedzy, a nie jako nieograniczony generator odpowiedzi. Najpierw sprawdzana jest wiedza wbudowana oraz fakty dodane przez użytkownika. Jeżeli baza nie obejmuje danego tematu, backend może pobrać streszczenie źródłowe i spróbować ocenić właściwość twierdzenia. Mechanizm rozpoznaje bezpośrednie wsparcie, bezpośrednie zaprzeczenia, relację stolicy oraz konflikt klas. Dzięki temu przykłady testowane po ostatniej poprawce działają następująco:

Twierdzenie	Wynik	Uzasadnienie
<i>Mars is a star</i>	fake	streszczenie klasyfikuje Marsa jako planetę, a nie gwiazdę
<i>Whales are fish</i>	fake	streszczenie klasyfikuje wieloryby jako ssaki
<i>Spiders are animals</i>	true	pająki są pajęczakami, a pajęczaki należą do zwierząt
<i>Paris is the capital of Germany</i>	fake	streszczenie wskazuje Paryż jako stolicę Francji
<i>covid is bacteria</i>	fake	WHO opisuje COVID-19 jako chorobę spowodowaną wirusem SARS-CoV-2

Granica pozostaje ostrożna: jeśli streszczenie nie daje jasnego wsparcia ani konfliktu, system nadal zwraca **uncertain**.

4.4 Oddzielenie news credibility od fact-checkingu

Artykuł informacyjny nie jest tym samym co twierdzenie typu “Słońce jest gwiazdą”. Dlatego dla modułu *News* wynik opiera się głównie na polu **credibility**, a nie na twardym **true/fake**. System wylicza:

- **source_score** – zaufanie do domeny;
- **article_quality_score** – jakość struktury artykułu;
- **corroboration_score** – potwierdzenia z niezależnych źródeł;
- **risk_score** – flagi ryzyka, np. brak daty albo słaba struktura.

Taki wynik jest uczciwszy: mówi, czy artykuł wygląda wiarygodnie, ale nie udaje pełnego dowodu logicznego.

4.5 Jak działa analiza obrazów AI

Moduł *Images* działa inaczej niż moduły tekstowe. Nie sprawdza twierdzenia i nie szuka artykułów potwierdzających. Backend przyjmuje plik graficzny, waliduje format i rozmiar, a następnie analizuje sygnały dostępne w samym pliku.

Najważniejsze etapy są następujące:

1. Plik jest sprawdzany pod względem typu i limitu rozmiaru.
2. Z obrazu odczytywane są metadane: format, wymiary, pola EXIF oraz proste informacje zapisane w pliku.
3. System szuka markerów generatorów AI w metadanych, np. `stable diffusion`, `midjourney`, `dall-e`, `comfyui`, `firefly`.
4. System szuka markerów w nazwie pliku, np. `ai-generated`, `imagefx`, `stable-diffusion`.
5. Jeżeli są obecne oryginalne pola aparatu, np. `Make`, `Model`, `LensModel`, `DateTimeOriginal`, wynik ryzyka jest obniżany.
6. Jeżeli nie ma metadanych aparatu, system dodaje ostrzeżenie, ale nie traktuje tego jako dowodu AI.
7. Analizowane są lekkie sygnały wizualne, np. kwadratowe rozmiary popularne w generatorach, gładkość obrazu, brak naturalnego szumu, korelacja kanałów koloru i wzór kompresji.
8. Opcjonalny duży model wizualny może być podłączony przez konfigurację, ale w domyślnej chmurze jest wyłączony, aby aplikacja działała szybko i bez płatnych zasobów.

Wynik `likely_ai` pojawia się wtedy, gdy znaleziono mocny marker generatora w metadanych lub nazwie pliku albo gdy lekki moduł wizualny widzi spójny zestaw sygnałów AI. Wynik `likely_not_ai` pojawia się przy mocnych sygnałach aparatu telefonu albo przy bardzo mocnym sygnale realnego obrazu z opcjonalnego modelu. W niejednoznacznych przypadkach system nadal zwraca `uncertain`, a UI pokazuje etykiety typu *Metadata missing*, *Weak file clues*, *Possible AI signals*, *AI metadata found*, *AI signals found* albo *Camera metadata found*.

To ograniczenie jest celowe. Zdjęcie z aparatu może zostać edytowane, zrzut ekranu może wyglądać jak zdjęcie, a obraz AI może zostać zapisany z fałszywymi metadanymi. Dlatego wynik modułu obrazów należy rozumieć jako analizę śladów pliku, nie jako pełny dowód autorstwa pikseli.

5 Architektura końcowa

5.1 Warstwy systemu

Warstwa	Najważniejsze pliki	Rola
Core engine	<code>src/factcheck/service.py</code>	Orkiestracja głównego pipeline'u i wybór strategii.
Decision layer	<code>src/factcheck/decision.py</code>	Decyzja <code>true/fake/uncertain</code> .
Retrieval	<code>src/factcheck/retrieval.py</code>	Pobieranie i dobór materiału źródłowego.
Common knowledge	<code>src/factcheck/common_knowledge.py</code>	Wbudowana i użytkowa baza wiedzy, proste fakty, reguły i lokalna wiedza.
Knowledge sources	<code>src/factcheck/knowledge_sources.py</code>	Wikipedia summary, klasy taksonomiczne, relacje stolic i oficjalna wiedza źródłowa.
Semantic NLI	<code>src/nli.py</code>	Opcjonalny model MNLI porównujący twierdzenie z dowodem jako para <code>premise/hypothesis</code> .
News credibility	<code>src/factcheck/news_credibility.py</code>	Punktacja artykułów informacyjnych.
Image/OCR	<code>src/factcheck/image_analysis.py</code>	OCR i ocena ryzyka obrazu AI.
REST API	<code>src/api_factcheck.py</code>	Publiczne endpointy dla Web UI i Fluttera.
Web UI	<code>src/ui.py</code>	Interfejs przeglądarkowy.
Desktop	<code>src/desktop_app.py</code>	PyWebView i lokalny serwer w aplikacji Windows.
Mobile	<code>apps/fake_news_detector_flutter/</code>	Aplikacja Flutter dla telefonu.
Cloud	<code>Dockerfile</code> , <code>render.yaml</code>	Wdrożenie backendu na Render.

5.2 Przepływ żądania

1. Użytkownik wpisuje URL, twierdzenie albo wybiera obraz.
2. Interfejs wysyła żądanie do API.
3. API waliduje dane wejściowe.
4. Silnik wybiera odpowiednią ścieżkę: `fact-check`, `common knowledge`, `news credibility` albo `image analysis`.
5. Wynik jest zwracany jako JSON.
6. UI renderuje krótką odpowiedź, szczegóły, źródła i ostrzeżenia.

5.3 Kontrakt API

Publiczny kontrakt zawiera cztery najważniejsze endpointy:

```
GET /health
GET /ready
POST /factcheck
POST /factcheck-image
GET /knowledge/facts
```

POST /knowledge/facts

Przykładowe pola wyniku:

```
verdict: true | fake | uncertain
confidence: liczba 0.0-1.0
summary: kr\'otkie wyja\'szenie
evidence: lista \'zr\'odeł{}
credibility: ocena wiarygodno\'sci news\'ow
image_analysis: OCR albo ocena ryzyka AI
trace: diagnostyka pipeline'u
```

Stabilność tego kontraktu jest ważna, ponieważ korzysta z niego aplikacja Flutter.

6 Implementacja interfejsów

6.1 Web UI

Interfejs webowy został zaimplementowany w `src/ui.py`. W aktualnej wersji widoczne są trzy sekcje:

- News;
- Facts;
- Images.

Sekcja *Screenshots* została usunięta z UI. Backendowa funkcja OCR nadal istnieje, ale nie jest osobnym modułem użytkownika. Po tej zmianie interfejs jest prostszy i lepiej odpowiada rzeczywistym zastosowaniom. W sekcji *Facts* dodano zwijany blok *Knowledge base*, dzięki któremu użytkownik może dopisać prosty fakt do bazy wiedzy bez edycji plików projektu.

6.2 Aplikacja desktopowa Windows

Wersja desktopowa nie jest osobnym produktem logicznym. Jest opakowaniem lokalnego Web UI w okno aplikacji Windows. Działa następująco:

1. `FakeNewsDetector.exe` uruchamia lokalny serwer Uvicorn.
2. Serwer publikuje `src.ui:app` na wolnym porcie localhost.
3. PyWebView otwiera lokalny adres w natywnym oknie.
4. Po zamknięciu okna serwer jest zatrzymywany.

Aktualna paczka desktopowa:

- `dist/FakeNewsDetector/FakeNewsDetector.exe`
- `dist/FakeNewsDetector-Windows-Portable.zip`

Po ostatniej przebudowie paczki desktopowej:

- EXE SHA256: DAEA040D5910CFADB38D756F6639469F
CDC54C3FAC74C713032571AE454201CF;
- ZIP SHA256: 30BD3B9AAADD897BD0ABFF0AF4AE9274
2D8AAA56299CB463B932A19F8B7A188A.

6.3 Aplikacja mobilna Flutter

Aplikacja mobilna jest klientem API. Nie uruchamia Pythona na telefonie. Schemat pracy wygląda tak:

telefon -> internet -> Render API -> wynik -> telefon

Pakiet Android:

- identyfikator: `app.veritylens.mobile`;
- główna aktywność: `app.veritylens.mobile.MainActivity`;
- publiczny backend: <https://fake-news-detector-poi6.onrender.com>;
- aktualny cloud APK po czystej przebudowie: `outputs/phone_download/VerityLens-cloud.apk`;
- APK SHA256: BE6B0BD8C9BFBF420B44EB3041D4417E
A6859F0CC4723BC5E4BEB5A580BDE112.

Telefon działa bez komputera, ale wymaga internetu i działającego backendu Render. Wersja darmowa Render może zasypiać, dlatego pierwszy request może trwać kilkadziesiąt sekund. To ograniczenie platformy hostingowej, nie błąd aplikacji.

6.4 Wybór oryginalnych zdjęć na Androidzie

Ostatnia poprawka mobilna dotyczyła modułu *Images*. Standardowy system wyboru pliku na Androidzie okazał się niewystarczający, ponieważ Galeria i Google Photos mogły oddawać aplikacji kopię obrazu pozbawioną EXIF. W takiej sytuacji backend nie mógł znaleźć pól aparatu, nawet jeżeli zdjęcie rzeczywiście zostało wykonane telefonem.

Finalnie dodano natywny kanał Flutter-Android:

`app.veritylens.mobile/original_image_picker`

Najważniejsze metody:

- `listCameraImages` – pobiera listę ostatnich zdjęć JPEG z `MediaStore` i `DCIM/Camera`;
- `loadCameraImage` – ładuje wybrane zdjęcie po identyfikatorze `MediaStore` ID;
- `pickLatestCameraImage` – szybka ścieżka do najnowszego zdjęcia aparatu.

W interfejsie Flutter przycisk *Camera originals* nie otwiera już zewnętrznego menu Androida. Zamiast tego pokazuje wewnętrzną siatkę zdjęć z miniaturami. Dzięki temu typowy scenariusz testowy działa tak:

1. użytkownik robi zdjęcie aparatem telefonu;
2. w aplikacji wybiera *Images*;
3. naciska *Camera originals*;
4. wybiera zdjęcie z listy oryginałów;
5. naciska *Check image*;
6. backend widzi oryginalne EXIF i może pokazać *Camera metadata found*.

7 Wdrożenie chmurowe

7.1 Render

Backend został wdrożony jako web service na Render. Render buduje i uruchamia aplikację z repozytorium GitHub lub z Dockerfile, dzięki czemu telefon może korzystać ze stałego publicznego API HTTPS. W projekcie zastosowano lekki runtime API:

- `requirements.api.txt`;
- `Dockerfile`;
- `render.yaml`.

Z paczki chmurowej wykluczono:

- stare modele treningowe;
- foldery `build/dist/outputs`;
- lokalne środowisko `.venv`;
- ciężki model wizualny AI-image detection.

Do runtime’u API dodano natomiast **transformers** do tokenizacji oraz wykorzystano obecny już **onnxruntime**, ponieważ moduł *Facts* używa lekkiego modelu NLI do semantycznego porównania twierdzenia z dowodem bez uruchamiania PyTorch na darmowym Render. Model jest ładowany leniwie: backend startuje normalnie, a model pobiera się dopiero przy pierwszym użyciu modułu faktów wymagającego NLI. Żeby darmowy Render nie wykonywał zbyt wielu drogich inferencji, liczba wywołań NLI i liczba analizowanych fragmentów dokumentu są ograniczone przez **FACTCHECK_MAX_NLI_CALLS** oraz **FACTCHECK_MAX_PASSAGES_PER_DOCUMENT**.

7.2 Dlaczego chmura była potrzebna

Telefon z aplikacją Flutter nie powinien wymagać komputera użytkownika. Dlatego sam APK nie wystarcza. Potrzebny jest publiczny backend HTTPS. Po wdrożeniu na Render telefon może wysłać żądanie z dowolnej sieci z dostępem do internetu.

8 Testy i dowody działania

8.1 Testy automatyczne

Aktualny stan po ostatnich pełnych kontrolach:

Zakres	Narzędzie / komenda	Wynik
Backend	<code>unittest</code>	136/136 OK
unit/integration		
Product acceptance	<code>run_product_acceptance.py</code>	93/93 OK
Flutter analyze	<code>flutteranalyze</code>	No issues found
Flutter tests	<code>fluttertest</code>	14/14 OK
Release gate	release gate	OK, 25 kroków w ostatnim trybie cloud/phone
Final cloud/phone	finalizer cloud-phone	OK
Desktop package	verifier desktopowy	OK

8.2 Acceptance benchmark

Zamiast deklarować ogólną skuteczność na całym internecie, projekt ma mierzalną bramkę regresyjną. Ostatni wynik acceptance benchmark:

Sekcja	Poprawne	Wszystkie	Skuteczność
Facts	44	44	100%
News	15	15	100%
Screenshot OCR API	12	12	100%
Images	11	11	100%
API/mobile contract	11	11	100%
Razem	93	93	100%

Należy to interpretować poprawnie: wynik 93/93 oznacza, że aktualny produkt przechodzi zdefiniowane przypadki kontrolne. Nie oznacza to matematycznej gwarancji 100% dla wszystkich możliwych tekstów i newsów w internecie. To jednak dobry fundament inżynierski, bo każda przyszła zmiana może zostać porównana z tym samym zestawem regresji.

8.3 Dowód działania na fizycznym telefonie

Fizyczny telefon został wykryty przez ADB jako:

RFCTC07YX2N, model SM-G781B

W końcowym wrapperze wymagano prawdziwego urządzenia, a nie emulatora. Raporty potwierdziły:

- urządzenie jest fizyczne;
- APK cloud jest w trybie release;
- w APK osadzono publiczny adres Render;
- telefon potrafi wykonać smoke test;

- zrzut ekranu jest poprawnym plikiem PNG;
- SHA256 zrzutu zgadza się z plikiem.

8.4 Paczka oddania

Wygenerowano paczkę:

`outputs/submission/VerityLens-submission.zip`

Dodatkowo po ostatnich poprawkach przygotowano czysty ZIP z kodem źródłowym:

`outputs/submission/VerityLens-source-code-final.zip`

Weryfikator paczki sprawdził:

- obecność raportów końcowych;
- obecność desktop ZIP;
- obecność APK;
- czysty ZIP źródła;
- brak zabronionych plików lokalnych lub wygenerowanych w źródłach;
- zgodność raportów z artefaktami.

Po końcowym audycie projekt miał status:

complete = true, estimated completion = 100%, remaining = 0%

9 Historia prac

Etap	Co robiono	Najważniejszy efekt
Analiza projektu	Przegląd istniejących plików, modeli i wejść API.	Wykryto konflikt między prototypem ML a produktem.
Refaktoryzacja	Utworzenie kanonicznego pipeline'u i archiwizacja legacy.	Jeden silnik dla API, UI, desktopu i telefonu.
Ostrożność decyzyjna	Wprowadzenie zasady uncertain .	System nie wymusza fałszywej pewności.
Facts	Dodanie reguł prostych faktów, arytmetyki i wiedzy lokalnej.	Lepsze odpowiedzi na stabilne codzienne twierdzenia.
Facts knowledge base	Dodanie endpointu <code>/knowledge/facts</code> , pliku <code>user_knowledge_base_v1.json</code> oraz formularzy w mobile i PC.	Użytkownik może rozszerzyć bazę wiedzy podczas demonstracji.
Facts source reasoning	Rozszerzenie interpretacji streszczeń Wikipedii i oficjalnych źródeł.	System odróżnia niezgodne klasy, np. planetę od gwiazdy i ssaka od ryby.

Facts NLI		Dodanie opcyjnego modelu Xenova/mobilebert-uncased-m na poziomie <code>uncertain</code> w trybie ONNX.	System potrafi semantycznie porównać twierdzenie z dowodem i rzadziej blokuje oczywiste przy-
News		Oddzielenie <code>credibility</code> od <code>hard verdict</code> .	Artykuły newsowe są oceniane jako wiarygodne/średnie/słabe, nie jako automatycznie prawdziwe albo fałszywe.
Images		Dodanie <code>risk assessment</code> dla AI obrazów.	Wynik mówi o ryzyku z metadanych i lek-kich sygnałów wizualnych, nie o absolutnym dowodzie pochodzenia pikseli.
Images meta-data		Kalibracja etykiet i komunikatów dla obrazów.	Zamiast stałego <i>Possible AI</i> aplikacja rozróżnia: <i>AI metadata found</i> , <i>Camera metadata found</i> , <i>Metadata missing</i> i <i>Weak file clues</i> .
Android originals	origi-	Natywny wybór oryginalnych zdjęć z aparatu.	Aplikacja omija zewnętrzną Galerię i pobiera zdjęcia z <code>DCIM/Camera</code> przez <code>MediaStore</code> .
Web UI		Zbudowanie oddzielnych paneli produktu.	Użytkownik może pracować przez przeglądarkę.
Desktop		PyInstaller + PyWebView.	Powstała paczka Windows portable.
Mobile		Flutter + API client.	Powstał APK Android.
Chmura		Render + Docker + release scripts.	Telefon może działać bez PC.
Dowody		Release gate, smoke tests, verifier, raporty.	Projekt ma powtarzalne potwierdzenie działania.
Uproszczenie UI		Usunięcie widocznej zakładki Screenshots.	Interfejs ma trzy jasne funkcje: News, Facts, Images.

10 Najważniejsze decyzje projektowe

10.1 Nie używać starego ML jako głównego produktu

Ta decyzja była kluczowa. Stary model mógł dawać szybkie etykiety, ale nie dawał zaufanego produktu. W projekcie weryfikacji informacji lepsza jest odpowiedź ostrożna i uzasadniona niż pewna, ale niekontrolowana.

10.2 Nie obiecywać globalnej dokładności 95% bez benchmarku

W rozmowie projektowej pojawiał się cel wysokiej jakości. Zamiast deklarować pustą liczbę, utworzono *product acceptance gate*. Dzięki temu 95% jest sprawdzane na konkretnych sekcjach i przypadkach, a nie tylko wpisane w opis.

10.3 Traktować telefon jako klienta, nie jako backend

Uruchamianie całego Pythona i OCR bezpośrednio na telefonie byłoby kosztowne i niestabilne. Dlatego aplikacja mobilna wysyła zapytania do chmury. To upraszcza APK, ale wymaga internetu.

10.4 Oddzielić demo od produkcji

Tryb LAN i tymczasowy tunnel są dobre do debugowania. Nie są jednak dowodem, że aplikacja działa sama na telefonie. Dlatego skrypty rozróżniają:

- LAN APK;

- internet tunnel APK;
- cloud APK release dla stałego Render URL.

10.5 Usunąć Screenshots z UI

Ostatnia decyzja produktowa dotyczyła uproszczenia. Funkcja screenshot OCR działa technicznie, ale jako oddzielna zakładka była mało użyteczna dla użytkownika końcowego. Została więc ukryta w API i testach regresyjnych, a interfejs końcowy ma tylko trzy czytelne moduły.

10.6 Nie udawać pełnego detektora AI obrazów

Moduł obrazów został nazwany i zaprojektowany jako szybka analiza ryzyka AI. To świadoma decyzja. W projekcie nie ma pełnej gwarancji rozpoznania każdego obrazu wygenerowanego przez AI na podstawie samych pikseli. Taka funkcja wymagałaby cięższego modelu, dobrego zbioru testowego, kalibracji progów oraz stabilnych zasobów obliczeniowych. W darmowej chmurze lepszym rozwiązaniem jest pokazać użytkownikowi konkretne ślady: metadane generatora, EXIF aparatu, brak EXIF, nietypowe wymiary, lekkie sygnały wizualne i ostrzeżenia. Dlatego interfejs mówi *Check image*, a nie *Prove AI image*.

10.7 Nazwać Facts bazą wiedzy

Po testach z wieloma krótkimi zdaniami przyjęto, że najuczciwszym opisem modułu *Facts* jest baza wiedzy z dodatkowymi mechanizmami źródłowymi. System może być rozszerzany przez użytkownika, ale nie powinien udawać, że zna każdy fakt świata. Jeżeli fakt jest w bazie, wynik jest szybki i wyjaśnialny. Jeżeli faktu nie ma, backend próbuje użyć źródeł, ale w razie słabych dowodów nadal zwraca *uncertain*.

10.8 Użyć NLI w faktach, ale nie robić z niego jedyne go sędziego

Moduł *Facts* potrzebował większej inteligencji niż proste porównywanie słów. Z tego powodu dodano model NLI, ale jego wynik nie jest traktowany jako absolutny wyrok. Model może pomylić się przy dwuznacznym zdaniach, skrótach, ironii albo słabych źródłach. Dlatego decyzja końcowa nadal zależy od jakości źródła, dopasowania fragmentu do twierdzenia, wyniku NLI i braku silnego kontrdowodu. To kompromis między użytecznością a ostrożnością: aplikacja ma być odważniejsza w oczywistych przypadkach, ale nie ma zgadywać bez dowodów.

11 Aktualny sposób działania

11.1 Na komputerze

Użytkownik może uruchomić:

- `dist/FakeNewsDetector/FakeNewsDetector.exe` – gotowa aplikacja desktopowa;
- `run_desktop_app.bat` – lokalny wrapper;

- `python -m uvicorn src.ui:app -port 8002` – Web UI;
- `python -m uvicorn src.api_factcheck:app -port 8001` – samo API.

11.2 Na telefonie

Telefon z aktualnym APK działa bez komputera, jeżeli ma internet. Schemat:

Verity Lens APK -> <https://fake-news-detector-poi6.onrender.com> ->
FastAPI -> wynik

Jeżeli Render Free zasnął, aplikacja może przez chwilę pokazać *API not reachable*. Po wybudzeniu serwera odświeżenie statusu powinno przywrócić połączenie. W module *Images* główna ścieżka wyboru zdjęcia korzysta z przycisku *Camera originals*. Ten przycisk pokazuje listę oryginalnych zdjęć z aparatu zamiast otwierać systemowe menu Galerii, które może zwracać kopie bez metadanych.

11.3 W repozytorium

Zmiany zostały wypchnięte na GitHub. Najważniejsze commity końcowe:

Commit	Znaczenie
7c57851	Początkowe uporządkowanie projektu Verity Lens.
fb97373	Finalizacja dowodów cloud/phone.
d5d0e16	Obsługa prostych faktów materiałowych.
ade5984	Kalibracja wiarygodności newsów.
892555a	Odświeżenie dowodów po kalibracji newsów.
b99006d	Usunięcie zakładki Screenshots z UI aplikacji.
0d14cff	Oficjalna terminologia WHO dla faktów COVID-19 i SARS-CoV-2.
e92ab8e	Ogólne rozumowanie źródłowo-taksonomiczne dla modułu Facts.
7e4799d	Przestawienie zakładki Images na sprawdzanie metadanych.
d68232f	Dodanie szybkiego wyboru najnowszego zdjęcia aparatu.
572bf9e	Próba wymuszenia systemowego pickera dokumentów dla oryginalnych plików.
114b2f7	Finalny wewnętrzny picker oryginalnych zdjęć z DCIM/Camera .
e1ee6f6	Dodanie rozszerzalnej bazy wiedzy w API, Flutterze i wersji PC.

12 Ograniczenia

- Backend w Render Free może zasypiać, więc pierwszy request może być wolny.
- Telefon nie działa offline, bo silnik działa w Pythonie po stronie backendu.
- Ocena AI obrazu jest oceną ryzyka na podstawie metadanych i lekkich sygnałów wizualnych, nie dowodem sądowym ani pełnym rozpoznaniem każdego generatora.
- Brak EXIF nie oznacza automatycznie AI, ponieważ metadane mogą zostać usunięte przez edycję, eksport, komunikator, Galerię albo serwis internetowy.
- Obecność EXIF aparatu nie gwarantuje pełnej autentyczności, ponieważ metadane można skopiować lub zmodyfikować.
- Pełny model wizualny AI-image detection nie jest domyślnie włączony w chmurze, ponieważ byłby ciężki dla darmowego hostingu i wymagałby osobnej kalibracji jakości.

- Model NLI poprawia rozumienie faktów, ale nie zastępuje źródeł i nie daje gwarancji poprawności dla każdego zdania.
- Fakty dopisane przez użytkownika są bazą demonstracyjną i powinny być traktowane jako wiedza lokalna, nie jako zewnętrznie zweryfikowany autorytet.
- Pierwsze użycie modułu faktów po uśpieniu lub wdrożeniu Render może być wolniejsze, ponieważ model NLI jest ładowany leniwie; liczba wywołań modelu jest ograniczona budżetem, aby nie przeciążyć darmowego backendu.
- OCR może się mylić przy zrzutach niskiej jakości, dlatego funkcja została wycofana z głównego UI.
- Obecny benchmark jest dobry jako regresja produktu, ale nie reprezentuje całego internetu.
- Aktualne fakty polityczne i urzędowe mogą się dezaktualizować, więc wymagają okresowej aktualizacji źródeł.

13 Możliwe dalsze prace

- dodać ekran *About / Version* w aplikacji mobilnej, aby od razu było widać commit, wersję APK i adres API;
- poprawić baner połączenia tak, aby przy Render Free pokazywał *Waking up server* i automatycznie ponawiał próbę;
- rozszerzyć benchmark acceptance o większy zbiór żywych newsów i trudnych przypadków;
- dodać import/eksport oraz panel przeglądania faktów dopisanych do bazy wiedzy;
- dodać osobny benchmark NLI dla krótkich twierdzeń i trudnych par dowód–twierdzenie;
- dodać opcjonalny płatny lub lokalny model wizualny do pełniejszej oceny obrazów AI;
- rozważyć obsługę Content Credentials / C2PA, jeżeli użytkownik będzie miał obrazy z podpisanym pochodzeniem;
- dodać tryb wykonania zdjęcia bezpośrednio w aplikacji, aby jeszcze mocniej kontrolować oryginalny plik i metadane;
- przygotować tryb demonstracyjny z zapisanymi przykładami wejściowymi;
- dodać historię ostatnich sprawdzeń w aplikacji mobilnej;
- rozważyć płatny hosting, jeżeli aplikacja ma działać szybko i stale.

14 Wnioski

Projekt zakończył się działającym produktem, a nie tylko prototypem. Najważniejszym sukcesem jest zmiana podejścia: od starego klasyfikatora *fake/real* do ostrożnego systemu weryfikacji informacji. Produkt ma jeden silnik, stabilne API, Web UI, desktop Windows, aplikację Flutter i backend w chmurze. Został przetestowany automatycznie i praktycznie, w tym na fizycznym telefonie. Moduł *Facts* został ostatecznie ustawiony jako rozszerzalna baza wiedzy, co jest prostsze do wyjaśnienia, testowania i obrony niż obietnica pełnej inteligencji ogólnej.

Najważniejsza lekcja projektowa brzmi: w systemach fact-checkingowych brak pewności nie jest porażką. Porażką byłoby udawanie pewności. Ta sama zasada dotyczy obrazów: projekt nie udaje, że darmowa wersja potrafi w pełni rozpoznać każdy obraz AI. Pokazuje konkretne ślady z pliku i jasno mówi, gdzie kończy się pewność. Dlatego *Verity Lens* jest zaprojektowany tak, aby pomagać użytkownikowi myśleć ostrożniej, a nie zastępować myślenie czarną skrzynką.

A Załącznik A: Komendy reprodukcyjne

```
python -m unittest discover -s tests -v
python scripts/run_product_acceptance.py
.\scripts\build_report.ps1
.\scripts\build_windows.ps1
.\scripts\verify_desktop_package.ps1
.\scripts\run_release_gate.ps1
.\scripts\build_phone_for_cloud.ps1 '
  -ApiBaseUrl https://fake-news-detector-poi6.onrender.com '
  -Mode release
.\scripts\make_source_bundle.ps1 '
  -OutputPath outputs\submission\VerityLens-source-code-final.zip
.\scripts\finalize_cloud_phone_submission.ps1 '
  -ApiBaseUrl https://fake-news-detector-poi6.onrender.com
flutter analyze
flutter test
adb devices
```

B Załącznik B: Najważniejsze artefakty

Artefakt	Ścieżka
Raport LaTeX	docs/report/verity_lens_report.tex
Raport PDF	docs/report/verity_lens_report.pdf
Desktop EXE	dist/FakeNewsDetector/FakeNewsDetector.exe
Desktop ZIP	dist/FakeNewsDetector-Windows-Portable.zip
Cloud APK	outputs/phone_download/VerityLens-cloud.apk
Source code ZIP	outputs/submission/VerityLens-source-code-final.zip

Submission ZIP	<code>outputs/submission/VerityLens-submission.zip</code>
Product acceptance report	<code>reports/product_acceptance_latest.json</code>
Release gate report	<code>reports/release_gate_latest.json</code>
Final cloud phone report	<code>reports/final_cloud_phone_submission_latest.json</code>

C Załącznik C: Skrócony opis dla uruchomienia

PC

Uruchomić:

`dist/FakeNewsDetector/FakeNewsDetector.exe`

Telefon

Zainstalować:

`outputs/phone_download/VerityLens-cloud.apk`

Telefon potrzebuje internetu. Backend działa pod adresem:

<https://fake-news-detector-poi6.onrender.com>